

实验报告：计算机视觉期中作业

姓名：李茗瑄

学号：24300980036

资源链接：

(1) Github Repo 链接：https://github.com/trapidation/CV_Midterm_Project

(2) 最优模型权重链接：

https://drive.google.com/drive/folders/10uflr_85MMmWWlkj1jdED17_AXvseVqw?usp=drive_link

一. 任务 1: 微调在 ImageNet 上预训练的卷积神经网络实现宠物识别

1.1 实验目的

本实验旨在通过对现有 CNN 网络输出层进行修改作为 baseline 进行训练和测试。训练时使用 ImageNet 预训练的网络参数进行初始化，从零训练输出层。同时，本实验通过修改训练步数，学习率，正则化参数，观察不同的超参数组合对训练准确度的影响，并与仅使用数据集从初始化开始训练得到的结果进行对比，观测预训练带来的提升。最后引入注意力机制进行训练并与 Baseline 结果进行对比。

1.2 实验配置

数据集来源：Oxford-IIIT Pet

训练集划分：采用 `split='trainval'` 作为训练集（包含 3680 张图像），采用 `split='test'` 作为验证/测试集（包含 3669 张图像）。

网络结构设计：

(1) Baseline 模型：采用 ResNet-18 架构，并将输出特征数由 1000 改成 37。

(2) 对比模型：采用轻量级 Vision Transformer 架构 ViT-Tiny，并同步加载预训练权重。

核心超参数配置：

Batch size: 32

初始 Epochs: 5 轮

优化器：Adam

差异化 Learning Rate: 特征提取层: $1e-4$ 新分类层: $1e-3$

正则化参数: 设定 L2 正则化惩罚项为 $1e-4$

编程框架：Pytorch

评价指标: 采用 `train_step_loss` 监控训练损失，采用 `val_accuracy` 检测验证集准确率。

可视化追踪: 采用 Swanlab 平台进行训练测试实时可视化。

1.3 模型结构

本实验主要由 `data_loader.py`，`train_baseline.py`，`train_scrach.py`，`train_sweep.py`，`train_vit.py` 这五个程序组成。

1.3.1 data_loader.py

本模块旨在通过 `datasets.OxfordIIITPets` 自动对目标数据集进行下载，并划分出训练集和测试集，对训练集进行随即裁剪和水平翻转，对测试集进行缩放统一规格。最终使用 ImageNet 的均值和标准差进行归一化，加速模型收敛。

1.3.2 train_baseline.py

本模块旨在采用 ResNet-18 作为 CNN 架构对输出层进行修改，通过 `model.fc = nn.Linear(num_fts, 37)` 将输出类型从 1000 改成宠物的 37 类，实现从零开始训练输出层。

1.3.3 train_scrach.py

本模块旨在完成模型预训练的消融实验。为了验证迁移学习在图像分类任务中的有效性，实验实例化了一个未加载任何 ImageNet 预训练权重的 ResNet-18 网络。模型的所有层参数均采用随机初始化方案。通过在 Oxford-IIIT Pet 数据集上从零开始进行训练，并将其损失函数下降曲线与验证集准确率同 Baseline 进行对比分析。

1.3.4 train_sweep.py

本模块旨在系统评估不同超参数及其组合对模型收敛与最终性能表现的影响。为确保实

验的严谨性，代码采用了控制变量法，并构建了自动化网格搜索逻辑，分别在不同的子项目中对以下三组超参数进行了独立测试。

(1) 学习率敏感度：设立(1e-5,1e-4,1e-3,1e-2)多组基准学习率，观察梯度下降步长对模型收敛速度的影响。

(2) 训练周期长短：在固定较优学习率和正则化参数的前提下，测试训练步长分别为 5, 10, 15 时对模型准确度的影响。

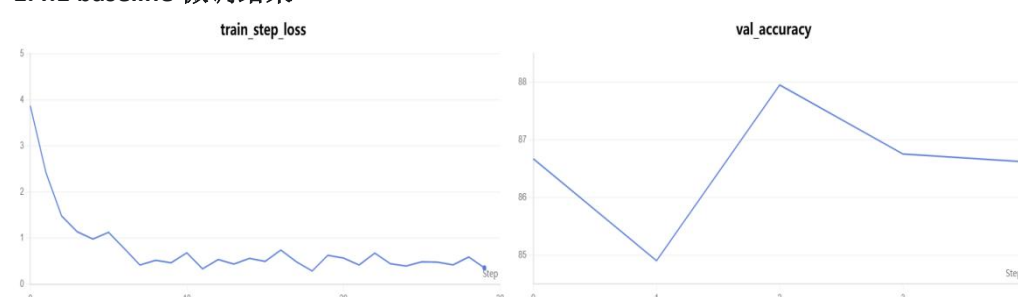
(3) 正则化惩罚：引入 L2 正则化惩罚对比无正则化，弱正则化，强正则化在抑制模型过拟合，和提升模型稳定性方面的影响。

1.3.5 train_vit.py

本模块旨在探究全局注意力机制在视觉分类任务中的表现。实验，借助 timm 算法库，引入了基于自注意力机制（Self-Attention）的前沿轻量级模型 Vision Transformer（具体采用 ViT-Tiny 架构）。实验保留了与 Baseline 一致的数据增强、批次大小及学习率分配策略，仅将最终的头（Head）线性层映射维度修改为对应的 37 个类别。

1.4 实验结论

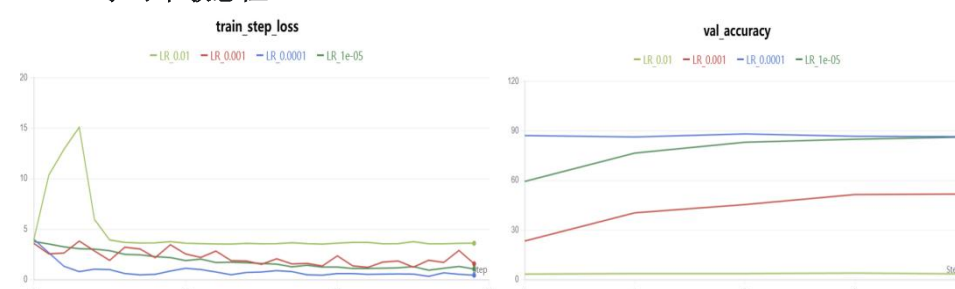
1.4.1 baseline 微调结果



实验首先在 ResNet-18 上进行了 Baseline 微调。如图 1 所示，模型在初始学习率为 1e-4 的设定下收敛迅速。经过 5 个 Epoch 的训练，训练 Loss 稳步下降，验证集准确率最终稳定在 86.62% 左右，证明了模型成功适配了 37 种宠物的分类任务。

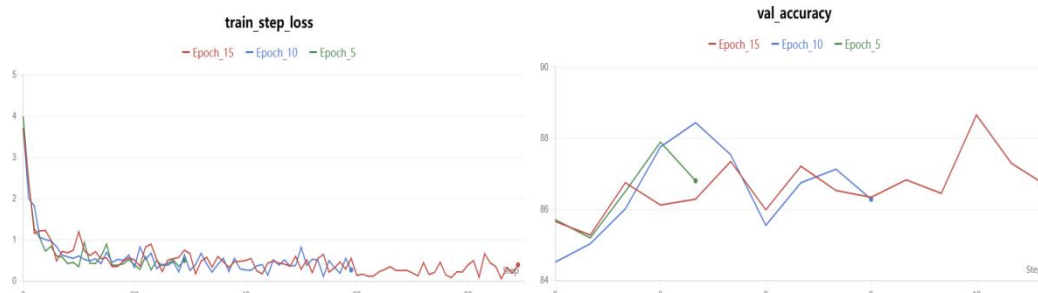
1.4.2 超参数敏感性分析

1.4.2.1 学习率敏感性



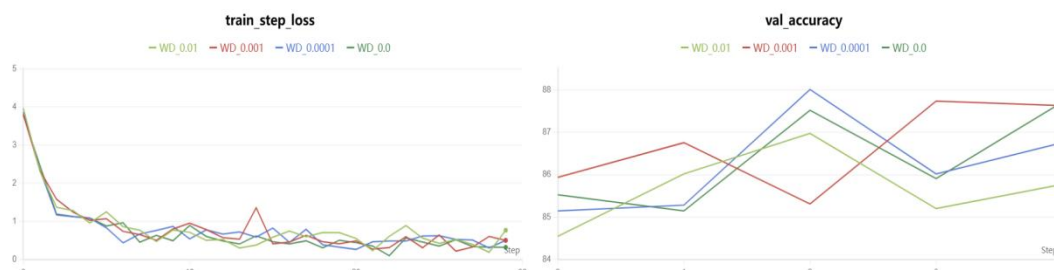
如图所示，对于学习率过大的情况(LR=0.01)不仅 Loss 在训练初期出现激增，且其验证集准确率几乎贴近于 0，这表明过大的学习率直接摧毁了原本 ImageNet 上的预训练权重。对于学习率较大的情况(LR=0.001),虽然 Loss 有所下降，但准确率起点极低，经过 5 轮训练后仅勉强爬升至 50% 左右，过大的步长跨越了最优解，导致模型在次优解附近徘徊。对于过小学习率(LR=1e-5)表现出了极其平稳的收敛态势，未出现任何震荡。然而，由于步长过小，其在 5 个 Epoch 内准确率仅从 60% 缓慢爬升至 80% 左右。若要达到最优性能，将需要消耗数倍的计算资源和训练轮数。最优学习率(LR=1e-4)表现出了极佳的拟合效率。其训练 Loss 迅速且平滑地降至接近 0 的水平；同时，验证集准确率在第一轮就直接跃升至近 88% 的高位，并在后续轮次中保持极其稳定的极高准确率。

1.4.2.2 训练周期



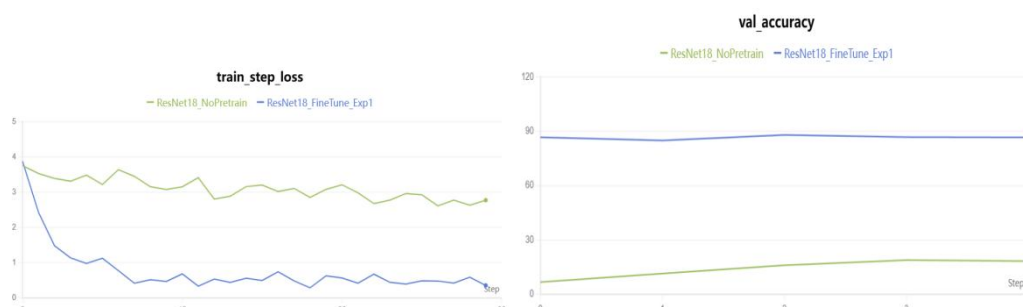
如图所示，三组模型在训练的极早期 Loss 值均出现断崖式下降，迅速从 4.0 降至 1.0 以下。这有力地证明了 ImageNet 预训练权重的强大特征提取能力，使得模型无需从零开始寻找梯度下降方向。但在训练中后期，Loss 曲线并未完全平滑，而是始终在 0.1 到 0.8 之间发生高频震荡。这可能是因为 ResNet-18 模型容量较大，且已经具备预训练先验知识，面对仅有 3000 余张图片的宠物数据集，模型在 4-5 个 Epoch 时实际上已经达到了拟合的最优点。为此针对此类迁移学习微调任务，较短的训练周期（如 5 轮左右）配合早停策略是最佳选择。

1.4.2.3 正则化惩罚



四组配置的训练 Loss 整体下降趋势基本一致，说明正则化项的引入并未破坏基础的梯度下降过程。但在微观层面上，强正则化（0.01，浅绿线）在后期的 Loss 略微偏高，而无正则化（0.0，深绿线）的 Loss 贴得最低。这符合理论预期：没有惩罚时，模型在训练集上拟合度最高。在基于 ResNet-18 的宠物分类微调任务中，完全不使用正则化会导致验证集震荡，而滥用强正则化会损害模型上限。采用弱正则化（Weight Decay = $1e-4$ ）是当前架构下的最佳折中方案，它能有效平滑优化地形，提升模型在未见数据上的最终泛化能力。

1.4.3 预训练消融实验



为了验证 ImageNet 预训练权重带来的收益，实验对比了随机初始化与预训练微调的模型表现。实验结果表明，两者具有显著差距，如图 1 所示，预训练部分在五步左右交叉熵损失便达到了 1 左右，而未使用预训练的模型交叉熵损失始终维持在 3 附近，下降缓慢。如图 2 所示，未使用预训练的模型在 5 轮训练后准确率仅停留在 20% 左右，远远低于预处理模型。这直观证明了：利用大规模数据集预训练获得的底层视觉特征提取能力，对于小样本下游任务（宠物分类）的快速收敛和性能提升具有决定性作用。

1.4.4 ViT 架构引入对比

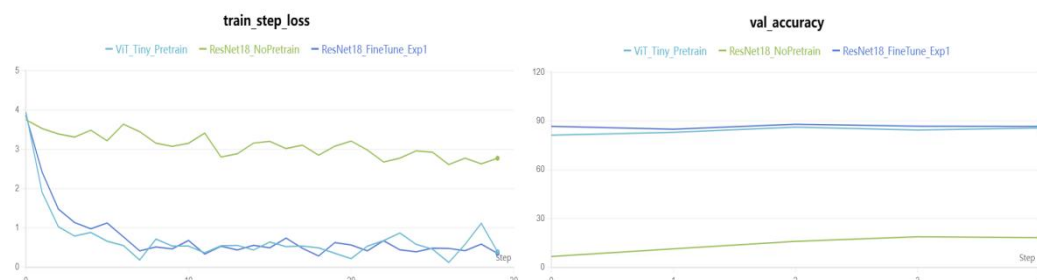


图 1 可以观察到，ViT_Tiny（浅蓝线）的初始下降斜率甚至比 ResNet-18 更加陡峭，其 Loss 更早地触底。这表明 Transformer 架构在全局特征的捕捉上具有极高的拟合效率。图 2 可以观察到，ViT_Tiny 展现出了极其优异的表现。尽管其初始起点（约 81%）略低于 ResNet-18，但其爬升稳健，最终准确率与 ResNet-18 高度重合，稳定在 86% 左右的极高水平。实验证明，即使是参数量极小的 ViT-Tiny 版本，在依托预训练的基础上，其特征表达能力也完全不输经典的 ResNet-18。

二.任务 2：场景目标检测与视频多目标追踪

2.1 实验目的

本实验旨在了解 YOLOv8 的架构特性，学会在复杂无人机航拍数据集（VisDrone）上进行定制化训练，提升模型对密集、小目标的特征提取能力。同时结合训练好的目标检测器与 ByteTrack 跟踪算法，实现对视频流的逐帧推理，理解卡尔曼滤波（Kalman Filter）与匈牙利匹配算法在维持目标 Tracking ID 上的作用机制。最后通过实际测试视频的逐帧观察，结合课上所学理论，分析物体遮挡、密集交汇场景下 Tracking ID 跳变与丢失的底层原因。

2.2 实验设置

数据集来源：VisDrone 无人机航拍数据集

基线模型：YOLOv8s

输入图像尺寸：640*640

Batch size：16

优化器：Adam

Epoch：20

编程框架：Pytorch

可视化：采用 Swanlab 平台进行训练测试实时可视化分析

2.3 模块设计

本任务的工程实现严格按照模块化思想进行设计，主要包含了三个核心代码脚本与两个关键视频文件，各模块的具体作用与生成逻辑如下：

2.3.1 核心代码设计

2.3.1.1 train.py

本模块旨在配置并自动下载 VisDrone.yaml 航拍数据集，设定相关的超参数（如 Epoch、Batch Size 等）启动本地端到端训练。同时，脚本内部集成了 swanlab 的回调函数接口，用于在训练过程中将 Loss 和 mAP 等指标实时同步至云端可视化面板。训练完成后，生成专属模型权重 best.pt。

2.3.1.2 count.py

本模块设计了基于帧间时空坐标对比的算法。通过维护一个字典 track_history 来存储每个对象在上一帧的中心点 Y 坐标（prev_y）。定义虚拟界线 LINE_Y，当且仅当满足 $prev_y < LINE_Y$ 且当前帧 $center_y \geq LINE_Y$ 时，触发计数器递增。

2.3.1.3 track_and_count.py

本模块首先加载 train.py 训练生成的 best.pt 专属权重；随后利用 OpenCV 读取 test_video.mp4 进行逐帧解析；调用 YOLOv8 内置的 ByteTrack 算法 (tracker="bytetrack.yaml") 为每个检测框分配稳定的 Tracking ID；接着嵌入 count.py 中的越线逻辑进行实时人数/车辆统计；最后，使用 OpenCV 的绘图 API 在画面上渲染 Bounding Box、ID 标签、红色警戒线及实时总数，并输出合成视频。

2.3.2 视频文件说明

2.3.2.1 test_video.mp4

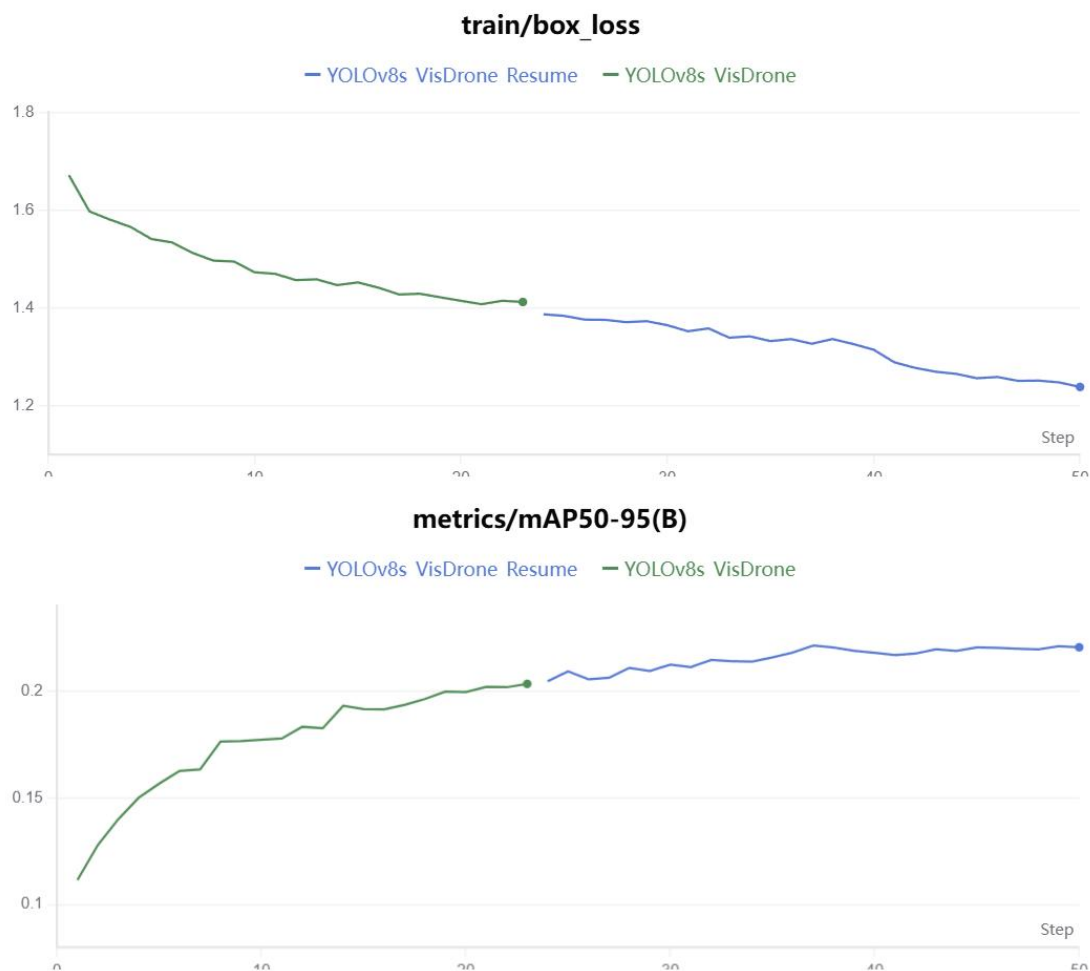
本视频由本人在现实场景（如天桥、高楼窗户等具有一定俯视角度的地点）使用手机实地拍摄录制而来。旨在模拟 VisDrone 数据集的无人机俯视视角，为模型提供未经见过的真实测试数据。视频内包含了密集的非机动车、行人交汇及自然遮挡等复杂场景，用于极限测试跟踪算法的鲁棒性。

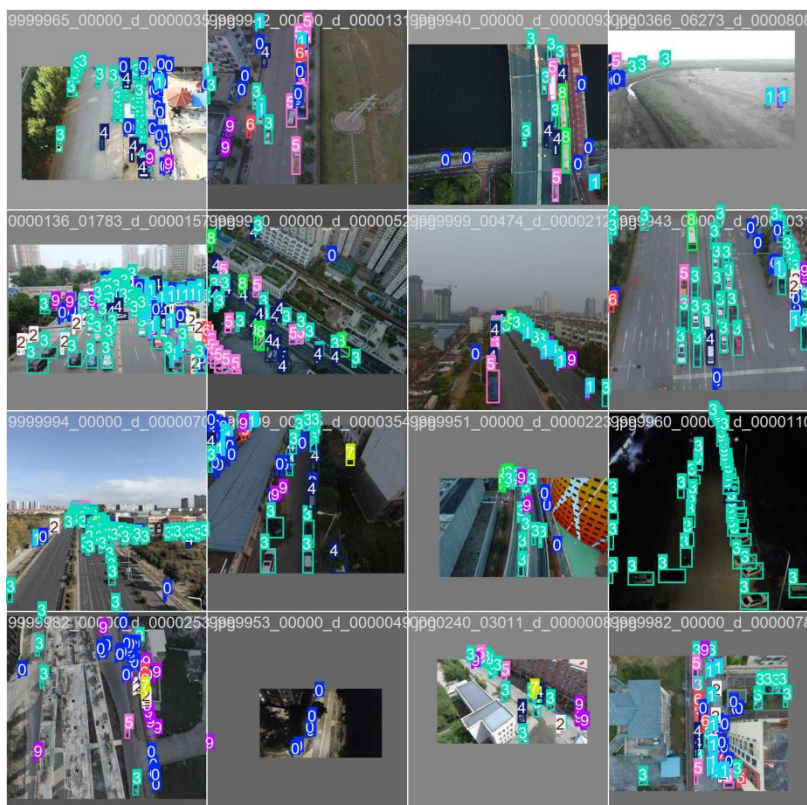
2.3.2.2 output_tracking.mp4

本视频由 track_and_count.py 脚本在完成所有推理与绘图操作后，通过 OpenCV 的 VideoWriter 自动逐帧拼接编码生成。它作为本次任务的最终可视化产物，直观展示了模型的检测效果与计数的准确性。

2.4 结果分析

2.4.1 模型训练评估





图一中曲线记录了模型在训练集上定位目标边界框的误差变化。从图中可以看出，初始阶段 Loss 值从约 1.68 快速下降，随后保持平稳的下降趋势，在第 50 轮时收敛至 1.25 左右。图中绿线与蓝线的无缝衔接，也验证了训练中途暂停并利用 **resume** 功能恢复训练的可靠性。这说明了损失值的平稳收敛，证明模型在逐步掌握提取无人机俯视视角特征的能力，对车辆、行人等目标的“定位 (Localization)”越来越精准，训练过程中未出现梯度爆炸或过拟合的异常震荡。

图二中 **mAP50-95** 是评估目标检测模型综合性能的严苛指标。图中显示，随着训练的推进，模型精度从最初的 0.11 稳步攀升，在 40 轮左右开始进入平台期，最终稳定在 0.22 左右。这可能是由于 **VisDrone** 是极具挑战性的航拍数据集（包含海量极小目标和密集遮挡），该 **mAP** 值的稳步提升说明模型已成功学习到了多类别的特征表达。后期曲线的平缓表明模型在当前的轻量级架构（YOLOv8s）与给定的超参数下，已达到了较好的收敛状态，具备了较强的“分类与检测”综合实力。

图三为预测结果可视化，从抽样的验证集预测结果 (Batch 0) 可以看出，模型在面临高速公路、密集路口、建筑工地以及暗光/黄昏等复杂场景时，均展现出了极高的鲁棒性。这说明模型能够紧密贴合地框选出极其微小的车辆与行人（如蓝色、粉色、青色框所示），这表明微调训练达到了预期目的。模型彻底摆脱了原始 **ImageNet** 视角的局限，成功将其能力迁移并适配到了无人机 (UAV) 高空俯拍的特殊应用场景中。

2.4.2 遮挡与 ID 跳变分析





效果检测：在图 1 中，模型对同一骑行者输出了两个重叠的检测框（id:2555 与 id:2444）。随着目标向下移动（图 2、图 3），id:2555 消失，ByteTrack 跟踪器成功维持了 id:2444。此时 id:2444 的中心点跨越红色虚拟线，越线逻辑正常触发，Total Crossed 准确地由 0 变为 1。冗余框的出现说明模型在复杂特征下置信度摇摆，NMS（非极大值抑制）未能完美过滤。但此时 ByteTrack 内的卡尔曼滤波工作正常，依靠平滑的运动轨迹预测，成功将跨线前后的 id:2444 进行了 IoU 关联。

出现问题：

（1）目标迷失：当骑行者继续向下移动，融入下方密集停放的自行车与电动车群中时（图 4），原本的白色框 id:2444 彻底消失。这可能是因为检测发生了严重的漏检（False Negative）。由于目标与背景（一堆停放的单车）在视觉特征上高度相似，构成了严重的语义遮挡和干扰，YOLOv8 检测器在该帧未能识别出移动的骑行者。在连续丢失观测值后，跟踪器被迫切断轨迹，将 id:2444 宣告死亡。

（2）ID 跳变：在 id:2444 消失的同时，画面中突然闪现出一个巨大的红色错误检测框（id:2610 tricycle，将骑行者和多辆停放的电动车框在了一起），同时背景 SUV 上也闪现了 id:2622。伴随着这些大框的出现，左上角的计数器错误地增加了 1，变成了 2。这可能是因为模型将杂乱的局部特征强行拟合，产生了误检（False Positive）。由于这个巨大红框的空间位置和尺寸与之前任何轨迹都不匹配（IoU 极低），跟踪器将其视为一个“全新出现”的目标，从而分配了全新的 id:2610。

总结：此片段有力地证明了，多目标跟踪的性能瓶颈往往在于前端的目标检测器。在密集、遮挡和背景干扰强烈的场景下，仅靠运动学信息（坐标、速度）和交并比（IoU）进行目标关联是非常脆弱的。一旦检测框发生闪烁、形变或漏检，ID 必然发生跳变。

三.任务 3：从零搭建与损失函数工程

3.1 实验目的

本实验旨在使用深度学习框架的基础 API，从零开始搭建语义分割网络 U-net。在无预训练权重的情况下复用 Oxford-IIIT Pet Dataset 实现像素级的三分类。并手动实现 Dice Loss，讨论标准交叉熵损失，Dice Loss 损失函数以及组合损失函数三种损失模型在验证集上 mIoU 的表现。

3.2 实验设置

数据集来源：Oxford-IIIT Pet

数据集划分：训练集 (trainval, 约 3680 张), 验证集 (test, 约 3669 张)

输入图像尺寸：使用 dataset.py 将图片尺寸 Resize 到 256*256

网络结构：4 层下采样自定义 U-net

Batch size:16

Learning Rate:3e-4

优化器：Adam

Epoch: 20

编程框架：Pytorch

可视化：采用 Swanlab 平台进行训练测试实时可视化分析。

评价指标：mIoU, Loss

3.3 模型结构设计

本实验主体由 `dataset.py`, `unet.py`, `loss.py`, `train.py` 以下四部分组成，分别负责 数据加载与预处理，网络结构搭建，损失函数工程，模型训练与评估。

3.3.1 dataset.py

本模块主要负责 Oxford-IIIT Pet 数据集的实例化、数据增强以及张量转换，它对输入图像统一执行 `Resize` 到 256×256 的操作，并使用 ImageNet 的均值和标准差进行归一化，以加快网络收敛速度。同时本模块在数据预处理阶段通过 `mask = mask - 1` 的逻辑，将其平移映射为严格的 $\{0, 1, 2\}$ 张量，避免了训练时的索引越界崩溃。

3.3.2 unet.py

本模块使用基础 API 从零构建 U-net 语义分割网络。网络包含 4 层最大池化下采样。输入张量尺寸由 256×256 逐层收缩至瓶颈层（Bottleneck）的 16×16 ，同时特征通道数由初始的 3 维逐次翻倍至 1024 维。此设计以牺牲空间分辨率为代价，极大提升了网络对全局高级语义信息的提取能力。在解码阶段，每进行一次反卷积上采样，都会通过 `torch.cat` 将编码器对应层级的高分辨率特征图在通道维度上进行拼接。此机制成功将低层级的“空间轮廓细节”与高层级的“抽象语义概念”相融合，是网络能够输出精细且无锯齿状分割掩码的关键所在。

3.3.3 loss.py

图像分割任务常面临严重的类别不平衡问题。为解决该问题，本模块自定义实现了三种损失计算逻辑：

交叉熵损失：调用内置的 `CrossEntropy_Loss`

Dice_Loss：依据 Dice 系数公式自定义实现。该函数首先通过 `Softmax` 将网络输出映射为概率分布，并将真实标签转化为 One-hot 编码。通过计算预测结果与真实标签的交集和并集来直接优化评价指标（IoU）。

组合损失：将上述两者线性相加，利用二者优势。

3.3.4 train.py

本模块是整合所有组件的调度中心，实现了完整的深度学习训练闭环，并具备以下工程规范

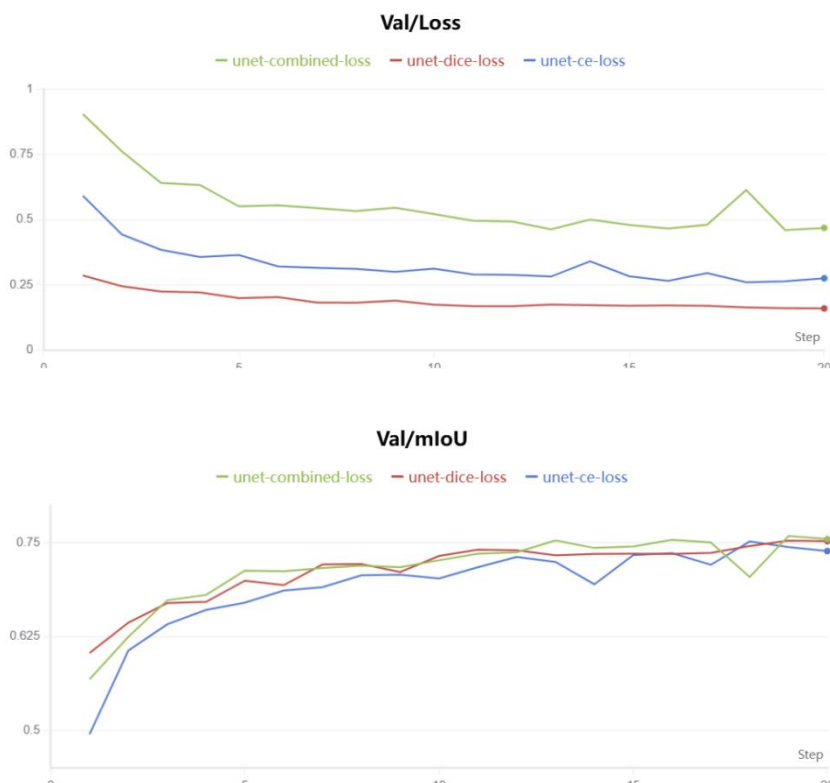
mIoU 评价指标构建：手动编写了计算平均交并比的评估逻辑。针对某些批次（Batch）中可能不存在特定类别（如某张图中没有边界像素）的情况，加入了动态过滤 NaN 的鲁棒性处理，确保指标计算的客观准确。

实验参数化与追踪：通过引入 `argparse` 命令行解析器，实现了无代码修改的损失函数切换。同时，深度集成了 Swanlab 实验追踪平台，在训练和验证循环中实时记录 Loss 下降趋势及 mIoU 攀升曲线。

3.4 损失函数与实验结果可视化及分析

3.4.1 可视化





3.4.2 实验结果分析

3.4.2.1 对于 Loss 分析

观察 Train/Loss 与 Val/Loss 曲线可以发现, Combined Loss(绿线)的绝对数值显著高于 CE (蓝线)和 Dice (红线)。但这并不意味着其模型收敛效果差。由于 Combined Loss 的计算逻辑为 $Loss_{ce} + Loss_{dice}$, 其数值在数学尺度上必然是两者的叠加。

3.4.2.2 对于 mIoU 分析与对比

仅使用交叉熵损失在训练初期起步最低(约 0.5), 虽然整体呈上升趋势, 但在训练周期, 其表现基本处于三者最下方, 最终 mIoU 约收敛在 0.74 左右。这可能是因为 CE Loss 是逐像素计算损失的。在牛津宠物数据集中, 存在严重的类别极度不平衡现象(背景像素量>宠物主体>宠物边界)。CE Loss 会为了迅速降低总体损失, 而去讨好占比极大的背景像素, 导致网络对细小的边界像素极度不敏感, 因此整体交并比(mIoU)提升缓慢且上限较低。

仅使用 Dice 损失在训练初期起步较高, 且在整个训练过程中上升非常平滑、稳定, 最终 mIoU 稳定在 0.75 左右, 显著优于单纯的 CE Loss。这可能是因为 Dice Loss 的本质是直接对评价指标(交并比)进行优化。它计算的是预测区域与真实区域的交集比例, 这种基于区域的计算方式天生对类别不平衡免疫。因此, 网络被强制要求关注那些面积虽小但至关重要的前景与边界像素, 从而获得了更高的分割精度。

对于组合损失, 绿线在中后期的表现最为亮眼, 并在最后几个 Epoch 中突破了红线和蓝线的瓶颈, 达到了全场最高的 mIoU(约 0.76)。值得注意的是, 绿线在第 18 个 Epoch 左右出现了一次明显的向下震荡, 但随后迅速反弹。这证实了 Combined Loss 完美结合了两者的优势: 在训练初期, CE 提供了稳定的逐像素梯度帮助网络快速找准大致方向; 在训练后期, Dice Loss 接管了对“难分样本”的精雕细琢。至于后期的轻微震荡, 可能是由于两种梯度的叠加导致在该阶段的有效学习率偏大, 但网络凭借强大的鲁棒性最终寻优到了更好的局部极小值点, 证明了组合损失是处理复杂语义分割任务的最优解。